LÖSUNGEN · LEHREREXEMPLAR

W3·3 – Teile und herrsche: Lösungen

Zum Arbeitsblatt `AQ_W3_AB3_Teile_und_herrsche.docx` · IDE-Level 3.3: siehe `AQ_W3_L3_Labyrinth_LSG.txt` (Klasse TeileUndHerrsche) · alle Tabellen, Bäume, Aufruf- und Vergleichszahlen mit Java (javac 21) nachgerechnet; binaerRek, maximumTH, mergeSort und zerlege identisch mit der Musterlösung · Rekursionstiefe: Erstaufruf = 0

Teil 1 (DS 21)

Aufgabe 0 – Das Spielkarten-Team

a) 15 Aufträge = 2 · 8 − 1 (1 Stapel mit 8, 2 mit 4, 4 mit 2, 8 Einzelkarten) – genau die Anzahl der Aufrufe von mergeSort für n = 8 (Java: 15). **b)** Mindestens **4** Vergleiche, wenn alle Karten eines Stapels kleiner sind ({1, 2, 3, 4} und {5, 6, 7, 8}: danach wird der Rest ohne Vergleich angehängt); höchstens **7**, wenn die Karten im Reißverschluss wechseln ({1, 3, 5, 7} und {2, 4, 6, 8}).

Aufgabe 1 – Das Orakel der Schatzkammer (9 BE)

Tiefe	lo	hi	mid	r[mid]	Rückgabe
0	0	10	5	30	7
1	6	10	8	51	7
2	6	7	6	38	7
3	7	7	7	45	7

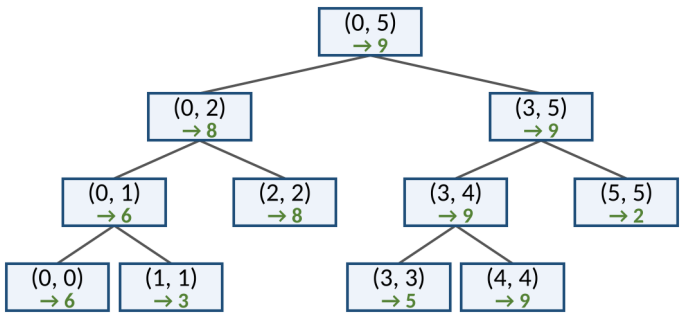
Tiefe	lo	hi	mid	r[mid]	Rückgabe
0	0	10	5	30	-1
1	0	4	2	13	-1
2	0	1	0	4	-1
3	1	1	1	9	-1
4	2	1	–	–	-1 (lo > hi)

a) links x = 45: Rückgabe **7** nach 4 Aufrufen; rechts x = 10: Rückgabe **-1** nach 5 Aufrufen. Die Rückgabe steht in jeder Zeile gleich (Endrekursion) (4 BE).

b) Bei n = 11 höchstens **5 Aufrufe** (erfolglose Suche: 4 Aufrufe mit nichtleerem Bereich + 1 Aufruf mit lo > hi; bei erfolgreicher Suche höchstens 4). Jeder Aufruf halbiert den Bereich mindestens: Nach etwa $\log_2 n$ Halbierungen bleibt ein Element. Höchstzahl $\lfloor \log_2 n \rfloor + 2$: n = 1000 → **11**, n = 1 000 000 → **21** (mit Java für alle Suchwerte bestätigt). Doppelt so viele Truhen kosten nur einen Aufruf mehr (3 BE).

c) Teilen: mid zerlegt den Bereich in r[lo … mid − 1], r[mid] und r[mid + 1 … hi]. **Herrschen:** Weil r sortiert ist, kann x nur in einer Hälfte liegen – nur sie wird rekursiv durchsucht, die andere verworfen; Basisfälle „gefunden“ und „Bereich leer“. **Zusammenführen:** Das Ergebnis des Selbstaufrufs wird unverändert zurückgegeben (2 BE).

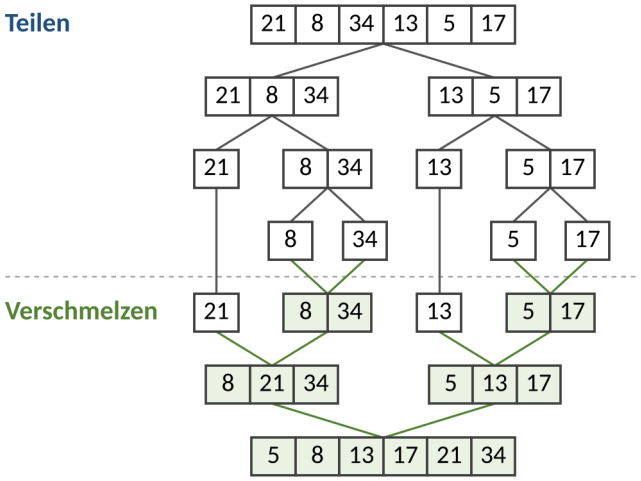
Aufgabe 2 – Die wertvollste Truhe (9 BE)



a) siehe Teilungsbaum, Rückgabe **9** (3 BE). **b) 11 Aufrufe**, größte Rekursionstiefe **3**, **5 Vergleiche** `links > rechts` (2 BE). **c)** Jedes Element ist genau ein Blatt (Basisfall von = bis) → n Blätter. Jeder andere Aufruf hat genau zwei Kinder; ein solcher Baum mit n Blättern hat n − 1 innere Knoten. Zusammen n + (n − 1) = 2n − 1 Aufrufe (Java für n = 1 … 16 bestätigt) (2 BE). **d)** Eine Schleife braucht ebenfalls n − 1 = 5 Vergleiche – Teile und herrsche ist hier **nicht** schneller, sondern hat zusätzlich 2n − 1 Aufrufe. Der Vorteil liegt woanders: Die

beiden Hälften sind unabhängig (parallel bearbeitbar), und dasselbe Muster macht Mergesort schnell, wo das Zusammenführen Arbeit spart (2 BE).

Aufgabe 3 – Truhen teilen, Truhen verschmelzen (11 BE)



a) Teilungsbaum: {21, 8, 34} | {13, 5, 17} → {21} | {8, 34} und {13} | {5, 17} → Einzelelemente. Verschmelzen: {8} + {34} → {8, 34} (1 Vergleich) · {21} + {8, 34} → {8, 21, 34} (2 Vergleiche) · {5} + {17} → {5, 17} (1 Vergleich) · {13} + {5, 17} → {5, 13, 17} (2 Vergleiche) · {8, 21, 34} + {5, 13, 17} → {5, 8, 13, 17, 21, 34} (4 Vergleiche) (4 BE).

i	j	k	Ergebnis bis jetzt
0	0	0	{}
0	1	1	{5}
1	1	2	{5, 8}
1	2	3	{5, 8, 13}
1	3	4	{5, 8, 13, 17}
2	3	5	{5, 8, 13, 17, 21}
3	3	6	{5, 8, 13, 17, 21, 34}

b) Tabelle: Startzeile und eine Zeile je übernommenem Element (3 BE). 4 Vergleiche (Zeilen 2 bis 5, solange beide Hälften Elemente haben), danach wird der Rest von a (21, 34) ohne Vergleich angehängt (letzte zwei Zeilen). **c)** mergeSort **11-mal** (2n − 1), verschmelze **5-mal** (n − 1, einmal je Aufruf, der kein Basisfall ist), insgesamt **10 Vergleiche** (1 + 2 + 1 + 2 + 4) (2 BE). **d)** Mit ≤: **(1, b), (2, d), (3, a), (3, c)**; mit <: (1, b), (2, d), (3, c), (3, a). Ein Verfahren ist **stabil**, wenn Elemente mit gleichem Schlüssel ihre ursprüngliche Reihenfolge behalten: (3, a) stand vor (3, c). Mit ≤ wird bei Gleichstand das Element der linken Hälfte zuerst übernommen → stabil; mit < das der rechten → nicht stabil (2 BE).

Aufgabe 4 – Level 3.3

Musterlösungen und häufige Fehler: `AQ_W3_L3_Labyrinth_LSG.txt`, Klasse TeileUndHerrsche (teilstueck mit bis exklusiv, mergeSort mit Kopie im Basisfall).

Teil 2 (DS 22)

Aufgabe 5 – Der Torwächter zerlegt die Schar (10 BE)

i	grenze	Reihung danach
–	0	6, 9, 3, 7, 1, 8, 4, 2
1	0	6, 9, 3, 7, 1, 8, 4, 2
2	1	6, 3, 9, 7, 1, 8, 4, 2 (tausche r[1] und r[2])

i	grenze	Reihung danach
3	1	6, 3, 9, 7, 1, 8, 4, 2
4	2	6, 3, 1, 7, 9, 8, 4, 2 (tausche r[2] und r[4])
5	2	6, 3, 1, 7, 9, 8, 4, 2
6	3	6, 3, 1, 4, 9, 8, 7, 2 (tausche r[3] und r[6])
7	4	6, 3, 1, 4, 2, 8, 7, 9 (tausche r[4] und r[7])
Ende	4	2, 3, 1, 4, 6, 8, 7, 9 (tausche r[0] und r[4])

a) Rückgabe **4**, **7 Vergleiche** (i = 1 bis 7); danach stehen 2, 3, 1, 4 links und 8, 7, 9 rechts vom Pivot 6 (4 BE).

Tiefe	lo	hi	p	Reihung nach zerlege	Vergleiche
0	0	7	4	2, 3, 1, 4, 6, 8, 7, 9	7
1	0	3	1	1, 2, 3, 4, 6, 8, 7, 9	3
2	2	3	2	1, 2, 3, 4, 6, 8, 7, 9	1
1	5	7	6	1, 2, 3, 4, 6, 7, 8, 9	2

b) Die beiden Aufrufe der Tiefe 1 sind Geschwister (dieselbe Tiefe); quickSort(r, 0, 0), (2, 1), (3, 3), (5, 5) und (7, 7) enden sofort und erscheinen nicht (4 BE). **c)** Insgesamt **13 Vergleiche** (7 + 3 + 1 + 2); Aufrufe von quickSort insgesamt **9** (4 zerlegende + 5 sofort endende), größte Tiefe 3 (2 BE).

Aufgabe 6 – Der schlechteste Tag des Torwächters (8 BE)

Tiefe	lo	hi	p	Reihung nach zerlege	Vergleiche
0	0	4	0	1, 2, 3, 4, 5	4
1	1	4	1	1, 2, 3, 4, 5	3
2	2	4	2	1, 2, 3, 4, 5	2
3	3	4	3	1, 2, 3, 4, 5	1

a) 10 Vergleiche = 4 + 3 + 2 + 1 (3 BE). Absteigend sortiert {5, 4, 3, 2, 1}: ebenfalls 10. **b)** Bei sortierter Eingabe ist das linke Pivot jeweils das **Minimum** des Bereichs: zerlege liefert p = lo, der linke Teil ist leer, der rechte hat nur ein Element weniger. Zerlegt werden Bereiche der Länge n, n − 1, …, 2 mit n − 1, n − 2, …, 1 Vergleichen: Summe n(n − 1)/2 (Java: n = 1 … 10: 0, 1, 3, 6, 10, 15, 21, 28, 36, 45). Größte Rekursionstiefe mit Zerlegung: **n − 2** (hier 3); mit den sofort endenden Aufrufen n − 1 (3 BE). **c)** Mergesort teilt unabhängig von den Werten immer in der Mitte – es gibt stets etwa $\log_2 n$ Ebenen. Abhilfe bei Quicksort: Pivot aus der Mitte, zufällig oder „Median aus drei“ (r[lo], r[mitte], r[hi]) wählen und vor dem Zerlegen nach vorn tauschen (2 BE).

Aufgabe 7 – Transfer: Der Drittel-Golem (12 BE)

Tiefe	Aufruf	r[0]	r[1]	r[2]	r[3]
0	golemSort(r, 0, 3)	5	8	1	3
1	golemSort(r, 0, 2)	3	8	1	5
1	golemSort(r, 1, 3)	1	3	8	5
1	golemSort(r, 0, 2)	1	3	5	8

a) Belegung nach dem Ende: **1, 3, 5, 8** (4 BE). Hinweis: Der Aufruf (0, 3) tauscht zuerst 5 und 3; die Belegung wird **zum Zeitpunkt des Aufrufs** notiert, also vor dessen eigenem Tausch. Vollständiger Verlauf mit allen 13 Aufrufen (zur Kontrolle):

Tiefe	Aufruf	Belegung beim Aufruf
0	golemSort(r, 0, 3)	5, 8, 1, 3

Tiefe	Aufruf	Belegung beim Aufruf
1	golemSort(r, 0, 2)	3, 8, 1, 5
2	golemSort(r, 0, 1)	1, 8, 3, 5
2	golemSort(r, 1, 2)	1, 8, 3, 5
2	golemSort(r, 0, 1)	1, 3, 8, 5
1	golemSort(r, 1, 3)	1, 3, 8, 5
2	golemSort(r, 1, 2)	1, 3, 8, 5
2	golemSort(r, 2, 3)	1, 3, 8, 5
2	golemSort(r, 1, 2)	1, 3, 5, 8
1	golemSort(r, 0, 2)	1, 3, 5, 8
2	golemSort(r, 0, 1)	1, 3, 5, 8
2	golemSort(r, 1, 2)	1, 3, 5, 8
2	golemSort(r, 0, 1)	1, 3, 5, 8

b) n = 2: **1**, n = 3: **4**, n = 4: **13** Aufrufe (A(n) = 1 + 3 · A(n − ⌊n/3⌋)). Wird n um die Hälfte größer, verdreifacht sich die Zahl der Aufrufe (n = 4 → 6 → 9: 13 → 40 → 121); Wachstum etwa n^2,7 – schlechter als n^2. mergeSort braucht für n = 9 nur 17 Aufrufe und etwa n · $\log_2$ n Vergleiche: golemSort ist korrekt, aber hoffnungslos ineffizient (3 BE).

c) Mit k = n/2 werden für n = 4 nur die Bereiche (0, 1), (2, 3), (0, 1) bearbeitet – sie **überlappen nicht**. Beispiel {3, 4, 1, 2}: Der erste Tausch liefert {2, 4, 1, 3}, danach sind beide Paare schon geordnet → Ergebnis **{2, 4, 1, 3}**, nicht sortiert (Java: 16 von 24 Permutationen falsch). Bei der Drittel-Teilung überlappen die beiden Zweidrittel-Bereiche um ein Drittel: Nach den ersten beiden Selbstaufrufen stehen die größten k Werte sicher im letzten Drittel, und der dritte Aufruf sortiert den Rest. Ohne Überlappung können Werte nicht von einer Hälfte in die andere wandern (3 BE). (Korrektheit der Originalfassung mit Java für alle Permutationen bis n = 7 geprüft: true.)

d) Selbstaufrufe gibt es nur für Bereiche mit mindestens 3 Elementen; dann ist k = n/3 ≥ 1 und jeder Selbstaufruf bearbeitet n − k < n Elemente. Die Bereichslänge sinkt also echt, bis höchstens 2 Elemente übrig sind – dort ruft sich golemSort nicht mehr auf (2 BE).

Aufgabe 8 – Der große Vergleich (8 BE)

Verfahren	schon sortiert	absteigend	schlechtester Fall	zusätzlicher Speicher	stabil?	Beispiel {6, 9, 3, 7, 1, 8, 4, 2}
Selectionsort	28	28	n(n − 1)/2	keiner (in-place)	nein	28
Insertionsort	7	28	n(n − 1)/2	keiner (in-place)	ja	23
Bubblesort (mit Abbruch)	7	28	n(n − 1)/2	keiner (in-place)	ja	28
Mergesort	12	12	≈ n · $\log_2$ n (n = 8: 17)	neue Teilreihungen, insgesamt ≈ n je Ebene	ja (mit ≤)	17
Quicksort (Pivot links)	28	28	n(n − 1)/2	keine Hilfsreihung, aber Aufrufstapel	nein	13

a) Zählweise wie in Welt 2 bzw. Level 3.3 (Insertionsort: jeder Vergleich beim Einordnen, Bubblesort mit Abbruch nach einem Durchgang ohne Tausch, Mergesort: Vergleiche in verschmelze, Quicksort: r[i] < pivot); alle Werte mit Java gezählt. Quicksort ist nicht stabil: Der Tausch mit grenze und der Schlusstausch des Pivots springen über gleiche Werte (5 BE). **b)** Im Mittel nur etwa n · $\log_2$ n Vergleiche mit sehr kleinem Aufwand je Schritt; arbeitet in-place ohne Hilfsreihungen; der schlechteste Fall lässt sich durch geschickte Pivotwahl praktisch vermeiden (2 BE). **c)** Bei kleinen oder fast sortierten Reihungen: im besten Fall nur n − 1

Vergleiche, kein Zusatzspeicher, stabil, einfach – deshalb schalten Bibliotheken bei kurzen Teilstücken auf Insertionsort um (1 BE).

Aufgabe 9 – Level 3.3 Experte

a) maximumTH = **9**; mergeSort liefert {1, 2, 3, 5, 6, 7, 8, 9}, Index von 8: **6**; quickSort: **17 Vergleiche**. Musterlösung von zerlege und quickSort sowie häufige Fehler: `AQ_W3_L3_Labyrinth_LSG.txt`.

Für Schnelle

Quickselect: Nach jeder Zerlegung wird nur ein Teil weiterbearbeitet; bei guten Pivots halbiert sich der Bereich: n + n/2 + n/4 + … ≈ 2n Vergleiche. **Median aus drei** macht sortierte Eingaben zum besten Fall. **Golem:** Aufrufzahlen für n = 1 … 12: 1, 1, 4, 13, 40, 40, 121, 121, 121, 364, 364, 364 – Potenzen-Muster (3^m − 1)/2.